

Самостійна робота №6: Node.js: поглиблене вивчення

Тема: Node.js: поглиблене вивчення

Мета: Поглибити розуміння внутрішніх механізмів Node.js - Event Loop, Streams, а також ознайомитися з базовим профілюванням продуктивності.

Кількість годин: 3

Теоретичний матеріал

Event Loop

Event Loop - центральний механізм асинхронності в Node.js. Node.js працює в одному потоці, але завдяки Event Loop може обробляти тисячі одночасних з'єднань. Event Loop складається з кількох фаз, які виконуються циклічно:

1. **Timers** - виконуються колбеки `setTimeout()` та `setInterval()`, чий час вичерпався.
2. **Pending callbacks** - колбеки системних операцій (наприклад, помилки TCP).
3. **Idle, prepare** - внутрішні операції Node.js.
4. **Poll** - отримання нових I/O подій, виконання I/O колбеків.
5. **Check** - виконуються колбеки `setImmediate()`.
6. **Close callbacks** - колбеки закриття (наприклад, `socket.on('close')`).

Microtasks та Macrotasks. Microtasks (Promise callbacks, `process.nextTick()`) мають вищий пріоритет та виконуються між кожною фазою Event Loop. `process.nextTick()` виконується раніше за Promise. Macrotasks - це `setTimeout`, `setInterval`, `setImmediate`, I/O операції.

Streams

Streams - абстракція для роботи з потоковими даними. Замість завантаження всього файлу в пам'ять, Streams обробляють дані частинами (chunks). Чотири типи:

- **Readable** - потоки для читання (`fs.createReadStream()`, `http.IncomingMessage`).
- **Writable** - потоки для запису (`fs.createWriteStream()`, `http.ServerResponse`).
- **Duplex** - двосторонні потоки (TCP socket).
- **Transform** - потоки, що трансформують дані (стиснення, шифрування).

Метод `pipe()` з'єднує потоки: `readableStream.pipe(writableStream)`.
 Pipeline (`stream.pipeline()`) - безпечна альтернатива з обробкою помилок.

Профілювання

`console.time()` / `console.timeEnd()` - простий вимір часу виконання. `--prof` флаг - вбудований профайлер V8, генерує лог, який аналізується через `--prof-process`. `perf_hooks` модуль - точний вимір продуктивності через `performance.mark()` та `performance.measure()`. **Clinic.js** - набір інструментів для діагностики: Clinic Doctor (загальний стан), Clinic Flame (flame graphs), Clinic Bubbleprof (асинхронні операції).

Контрольні питання

1. Перелічіть фази Event Loop та опишіть, що виконується на кожній з них.
2. Яка різниця між microtasks та macrotasks? Який порядок їх виконання?
3. Чим відрізняється `process.nextTick()` від `Promise.resolve().then()`?
4. Що таке Streams і яку проблему вони вирішують порівняно з буферизацією?
5. Назвіть чотири типи Streams та наведіть приклад використання кожного.
6. Як працює метод `pipe()` та чому `stream.pipeline()` є кращою альтернативою?
7. Які інструменти профілювання доступні у Node.js?

Практичне завдання

1. Напишіть скрипт, що демонструє порядок виконання: `setTimeout`, `setImmediate`, `Promise.resolve().then()`, `process.nextTick()`.

Поясніть результат.

2. Реалізуйте копіювання великого файлу (>100 MB) за допомогою Streams з прогресом виконання у консолі.

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- [Node.js - Event Loop](#) - офіційний гайд по Event Loop
- [Node.js - Streams](#) - API документація Streams
- [Node.js - Performance Hooks](#) - API для профілювання
- [Understanding the Node.js Event Loop - візуалізація](#) - візуальний гайд по Event Loop
- [Clinic.js](#) - інструменти для діагностики Node.js
- [Node.js Streams - документація](#) - практичний гайд по Streams та backpressure